

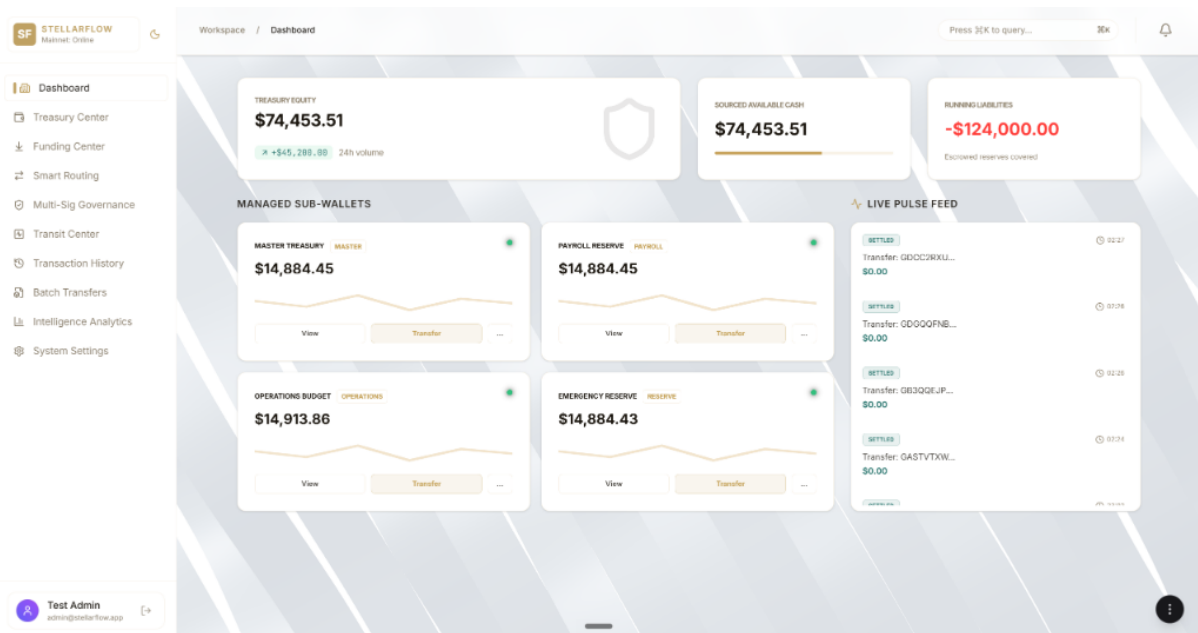
StellarFlow

Enterprise Treasury Operating System

Developer: Bibek Das

Project Overview

StellarFlow is a production-grade Web3 dashboard designed to solve the Just-In-Time (JIT) Liquidity problem for corporate treasuries. It aggregates funds proportionally from multiple isolated wallets and routes them through a single atomic transaction on the Stellar network.



Key Project Links

- **GitHub Repository:** <https://github.com/Be-bibek/StellarFlow>
- **Live Demo:** <https://web3-private-production.up.railway.app/>
- **Smart Contract (Testnet):** `CC7RCRZ3JF3W2YNTQKYTRMVGVI ZGLPIX6B2R7Q6HUOWDRK3IQKRQWLKT`

Tech Stack

- **Frontend:** Next.js (React), Tailwind CSS, TypeScript
- **Backend:** Rust (Actix-Web), PostgreSQL, WebSockets
- **Blockchain:** Stellar (Soroban Smart Contracts), Stellar SDK

System Architecture



Core Deliverables Completed

☒ Multi-Wallet Architecture Integration

- The application natively supports connecting with multiple wallet providers (Freighter, xBull, and others), breaking vendor lock-in and allowing diverse corporate actors to interact securely.

☒ Advanced Smart Contract Deployment

- A highly optimized Soroban smart contract is deployed on the Stellar Testnet. It acts as an on-chain router, instantly aggregating balances across multiple treasury vaults for large payouts.

☒ Balance & Real-Time Transaction Tracking

- A comprehensive transaction history and balance verification dashboard allows users to monitor live account states. The UI reflects on-chain changes instantly via a Rust-powered WebSocket event stream.

☒ Automated CI/CD Pipeline

- The project utilizes an automated Continuous Integration/Continuous Deployment (CI/CD) pipeline. This ensures that all code and smart contract logic is rigorously tested before hitting production.

☒ Production-Ready Documentation & Presentation

- The repository includes deep architectural planning documents, setup instructions, and code comments designed for enterprise handoff and rapid developer onboarding.

☒ Robust Error Handling

- The frontend cleanly intercepts all edge cases (Wallet Rejections, Simulation Failures, Contract Reverts) and surfaces them as user-friendly notifications.